ACADEMIA DE STUDII ECONOMICE BUCUREȘTI

FACULTATEA DE CIBERNETICĂ, STATISTICĂ ȘI INFORMATICĂ ECONOMICĂ

Proiect Cloud Computing

CatAdopt –
Platforma Web pentru adoptia de pisici

Master: Sisteme informatice pentru managementul proceselor și resurselor economice

Student: Neagu Corina-Maria

Grupa: 1146

Bucuresti, 2026

Link video:

Link publicare:

Deploy Vercel **https://cloud-computing-tan.vercel.app/**

Github **https://github.com/CorinaNeagu/CloudComputing**

1. Introducere

Tehnologiile Cloud au transformat modul in care interactionam cu mediile online, oferind atat scalabilitate cat si accesibilitate globala. Proiectul CatAdopt reprezinta o platforma web dedicata facilitarii procesului de adoptie a animalelor, construita pe o arhitectura de tip Cloud-Native.

Obiectivul principal al acestui proiect este demonstrarea modului in care o suita de servicii Cloud pot fi integrate pentru a crea o solutie completa, sigura si eficienta. Aplicatia utilizeaza Next.js pentru frontend si backend, MongoDB Atlas pentru persistenta datelor, Cloudinary pentru gestionarea continutului media, SendGrid pentru comunicarea automatizata cu utilizatorii si Google Open Authentication pentru autentificarea utilizatorului. Aceasta abordare elimina necesitatea gestionarii unei infrastructuri fizice, devenind Severless si permitand dezvoltatorului sa se concentreze si pe experienta utilizatorului.

2. Descriere problemă

Procesul traditional de adoptie a animalelor se confrunta adesea cu lipsa centralizarii, dificultati in monitorizarea cererilor, comunicare ineficienta intre partile implicate si riscuri de securitate. Solutia propusa abordeaza aceste probleme si ofera un punct central de interactiune digitala unde statusul fiecare pisici este vizibil in timp real, cererile sunt gestionate automat prin dashboard-ul utilizatorului, iar notificarile prin e-mail asigura o legatura directa si rapida intre partile implicate, toate acestea fiind sustinute de o infrastructura Cloud care garanteaza disponibilitatea serviciului 24/7.

3. Descriere API

Pentru a dezvolta o aplicatie cu o arhitectura moderna de tip Cloud-Native, am implementat si am integrat patru solutii Cloud distincte: MongoDB Atlas pentru gestionarea bazelor de date NoSQL, Google OAuth 2.0 pentru securizarea identitatii utilizatorilor, Cloudinary pentru optimizarea si livrarea continutului media si SendGrid pentru infrastructura de comunicatii prin e-mail. Aceasta abordare decuplata permite aplicatiei sa functioneze in regim Stateless, delegand responsabilitati tehnice distincte si complexe catre furnizori de servicii specializate precum Software-as-a-Service, Database-as-a-Service si Email-as-a-Service.

3.1. MongoDB Atlas

MongoDB Atlas este un serviciu de tip Database-as-a-Service care reprezinta componenta de stocare a datelor aplicatiei, fiind o solutie de tip NoSQL gazduita integral in Cloud. Datele sunt gazduite in Cloud iar instrument de tip GUI care ne permite sa interactionam cu baza de dare este MongoDB Compass.

Un aspect important pe care l-am configurat a fost IP Access List. Aceasta este o masura de securitate care ii spune bazei de date sa accepte conexiuni doar de la serverele unde este gazduita aplicatia. Am setat accesul de la adresa 0.0.0.0/0 dintr-un motiv tehnic: aplicatia ese gazduita pe Vercel, care utilizeaza o infrastructa serverless si foloseste adrese IP dinamice care se schimba constant din motive de securitate. Deschiderea accesului a fost necesara pentru a asigura comunicarea neintrerupta intre serverul aplicatiei si baza de date. Desi accesul este permis de la orice IP, securitatea este mentinuta printr-un Connection String, un sir de caractere securizat care include credentialele de acces. Acest URL de conexiune este stocat in variabila de mediu NEXT_DATABASE_URI. In arhitectura proiectului, utilizarea acestor variabile de mediu este esentiala pentru securitate permitand pastrarea cheilor de acces si parolelor in afara codului sursa.

3.2. Google OAuth 2.0

Acest serviciu este utilizat pentru gestionarea identitatii utilizatorilor fara a stoca parole in baza de date proprie prin protocolul OAuth 2.0 combinat cu middleware-ul NextAuth.js. In figura 1 se poate observa configurarea Client ID-ului prin care utilizatorul nu interactioneaza direct cu codul din proiect cand se logheaza, ci cu serverele Google. Acest lucru este posibil prin Client Secret care este cheia privata care permite serverului proiectului sa comunice in timp real cu serverul Google.

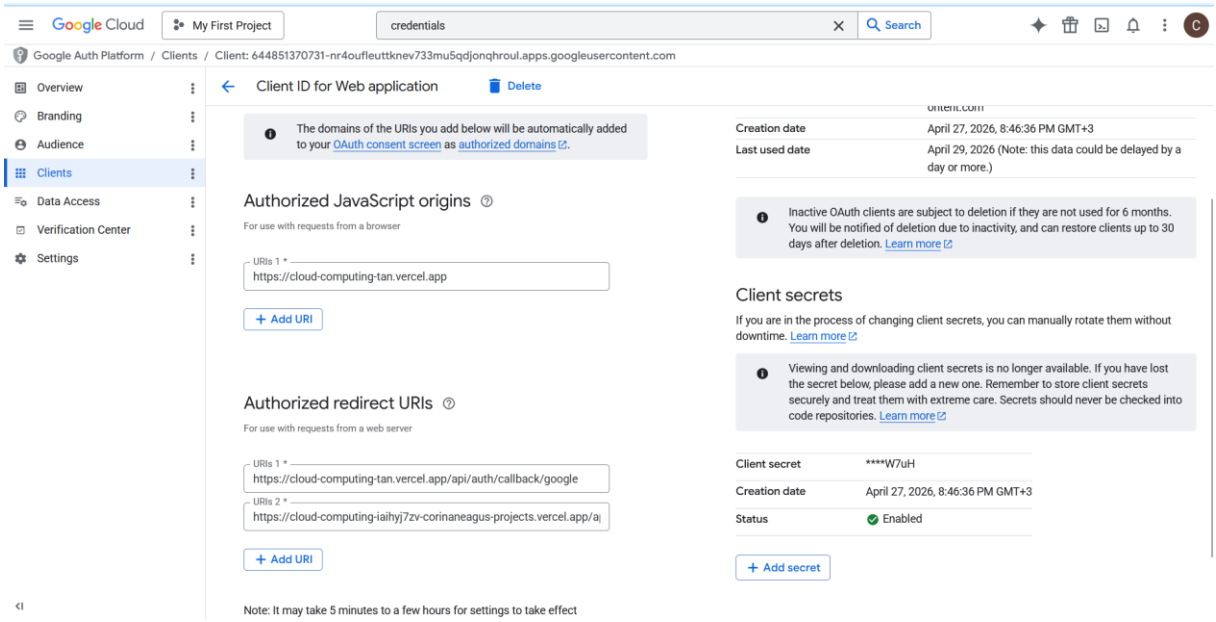

Figure 1. Google Cloud Console

De asemenea, se poate observa si masura de securitate care ii spune serverului Google sa accepte cereri de autentificare doar de pe domeniul aplicatiei. Aceasta se numeste Authorized JavaScript Origins iar prin configurarea domeniului a fost creat o lista alba (Whitelist) care sa blocheze procesul de autentificare de pe orice alt site neautorizat. Practic, daca un hacker ar reusi sa obtina Client ID-ul aplicatiei, Google ar identifica imediat ca cererea vine de la un alt domeniu si ar respinge conexiunea. Odata ce Google confirma identitatea, este trimis un cod securizat catre un endpoint specific in Authorized Redirect URI unde NextAuth.js il receptioneaza. NextAuth.js este un middleware care preia codul, extrage informatiile profilului si creaza sesiunea de lucru. Folosind acest sistem, aplicatia ramane Stateless adica nu trebuie inregistrat cine e logat pe server deoarece NextAuth.js foloseste un token nou de acces de fiecare data cand cineva se logheaza, token pe care utilizatorul il poarta cu el in browser sub forma unui cookie. Astfel, arhitectura transmite responsabilitatea securitatii aplicatiei catre infrastructura Google.

3.3. Cloudinary

Acest API functioneaza ca un serviciu de stocare si optimizare media de tip SaaS. Atunci cand un utilizator incarca o poza, aceasta nu este salvata in baza de date, ci in cloud-ul Cloudinary. Poza este stocata sub forma unui URL in baza de date, iar accesandu-l utilizatorul va fi redirectionat catre un browser care va solicita imaginea direct de la serverele Cloudinary. Am ales sa nu stochez imaginile direct in baza de date pentru a mentine aplicatie rapida si scalabila intrucat gestionarea fisierelor binare ar duce la degradarea performantei interogarilor si ar creste costurile.

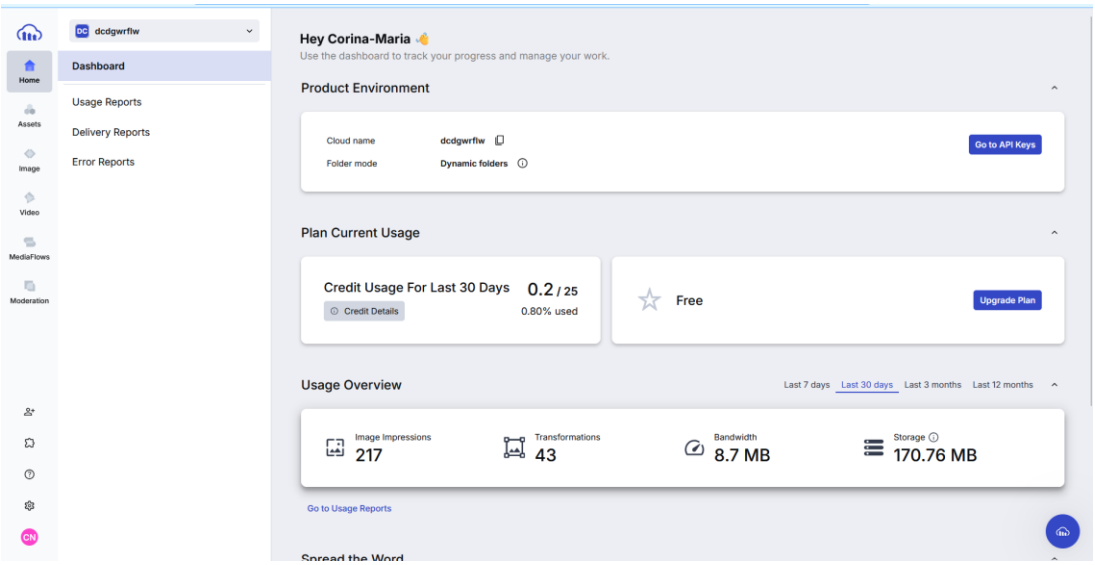

Figure 2. Cloudinary Dashboard

In figura 2 se poate observa dashboard-ul Cloudinary asociat aplicatiei CatAdopt:

- Sectiunea de Product Environment contine identificatorul unic in infrastructura cloud a serverului, adica acel Cloud Name dcdgwrflw, care se regaseste si in variabila de mediu CLOUDINARY_CLOUD_NAME din fisierul .env din proiect. Acest identificator este modul prin care cloud-ul separa datele utilizatorilor intre ele, chiar daca toti folosesc acelasi hardware, adica acelasi server, fiind integrat in URL-ul fiecare imagini pentru a redirectiona traficul catre spatiul de stocare privat si pentru a izola spatiul personal de stocare fata de ceilalti utilizatori ai platformei..
- Cloudinary masoarea utilizarea prin „credite”. Pentru aplicatie am folosit doar 0.80%, ceea ce arata ca arhitectura este eficienta
- Datele reale se pot vedea in sectiunea Usage Overview. Image Impressions arata faptul ca pozele din aplicatie au fost afisate de 217 ori, acest numar crescand de fiecare data cand un utilizator intra pe pagina sau in carusel. Transformations este o functie inteligenta a cloud-ului care arata ca 43 de imagini au fost redimensionate sau optimizate automat pentru a se potrivi in grid-ul de pe pagina principala sau in caruselul cu poze de pe homepage-ul utilizatorul nelogat. Bandwith reprezinta traficul de date consumat pentru a livra pozele de pe serverele Cloudinary catre browserul utilizatorului, iar valoarea de 8.7 MB confirma ca imaginile sunt bine optimizate. Storage reprezinta spatiul ocupat de toate pozele incarcate pana la momentul in care a fost facuta captura de ecran.

3.4. SendGrid

Acest serviciu este utilizat ca o solutie de tip SaaS pentru gestionarea si livrarea e-mailurilor din cadrul aplicatiei. In loc sa folosim un server de mail propriu, a fost delegata sarcina catre SendGrid. In aplicatie, API-ul este activat in momente cheie, cum ar fi confirmarea unei cereri de adoptie sau notificarea unui utilizator. Acest lucru este posibil prin intermediul unui API Key, o cheie securizata stocata in variabila de mediu SENDGRID_API_KEY, care permite aplicatiei sa dea comenzi serverlor SendGrid pentru a trimite mesaje in numele nostru.

In figura 3 se poate observa Dashboard-ul care ofera informatii in timp real despre fluxul de mesaje:

- Requests & Delivered: au fost efectuate 70 de solicitari de trimitere e-mail din aplicatie iar toate au fost livrate cu succes. Acest lucru confirma integrarea prin variabila de mediu SENDGRID_API_KEY este corecta si ca serverele Google accepta mesajele ca fiind sigure.
- Opened: metrica de 137.14% arata ca cele 96 de deschideri depasesc numarul de mailuri trimise. In contextul Cloud, acest lucru indica faptul ca utilizatorii au revenit asupra e-mailurilor primite de mai multe ori.
- Bounces & Spam Reports: Faptul ca sunt 0 mail-uri respinse sau raportate ca spam este crucial intrucat demonstreaza ca infrastructura aleasa este una de incredere si ca aplicatia respecta normele de trimitere a mesajelor.
- Reputation: In coltul stanga jjos se observa scorul de reputatie 100%, care garanteaza ca notificarile noastre ajung in Inbox-ul utilizatorului, nu in alte foldere.

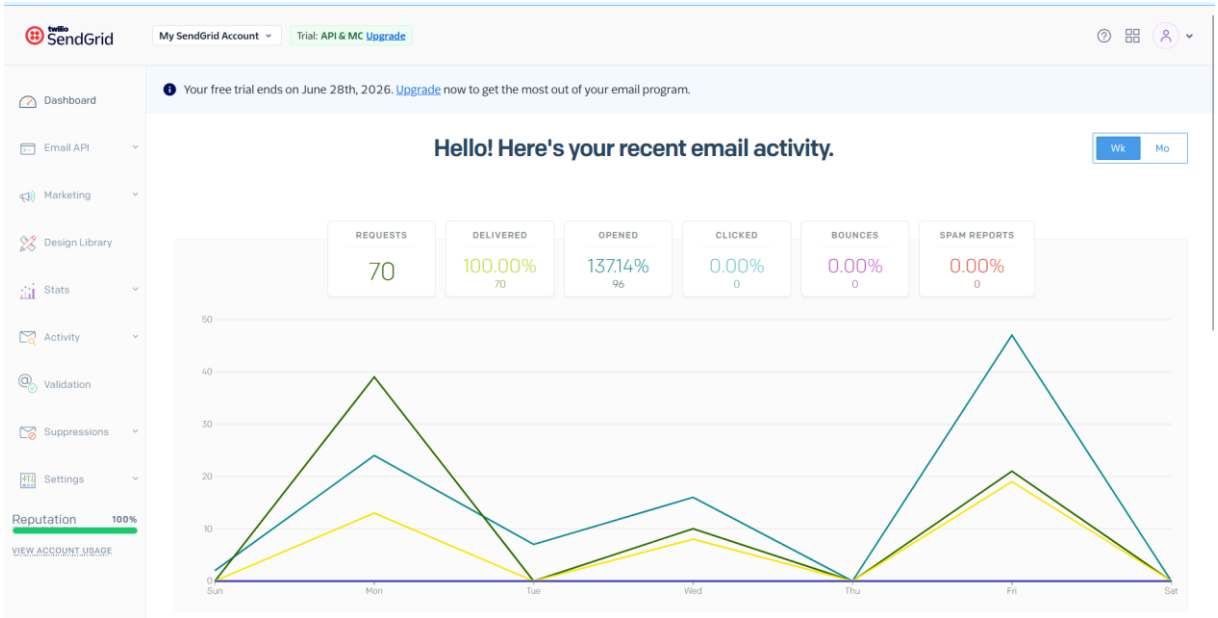

Figure 3. SendGrid Dashboard

4. Flux de date

4.1. Metode HTTP si Endpoint-uri

Aplicatia comunica intre frontend si backend folosind metodele standard REST:

Metoda HTTP	Endpoint	Descriere
GET	/api/cats	Preia lista tuturor pisicilor din MongoDB
POST	/api/adoption	Trimite o cerere de adoptie
PATCH	/api/requests/[id]	Modifica un singur camp, anume statusul cererii (Accepted/Rejected)
DELETE	/api/cats/[id]	Șterge un anunț din sistem (permis doar proprietarului).
POST	/api/send-email	Declanseaza serviciul SendGrid pentru notificari
PUT	/api/cats/[id]	Inlocuieste complet datele unui anunt existent

4.2. Exemple de Request / Response

Pentru a valida logica serverului independent de interfata utilizatorului, am utilizat Postman. Acesta mi-a permis sa simulez cereri catre API si sa verific daca serverul comunica corect cu MongoDB Atlas si returneaza datele in formatul JSON asteptat de interfata utilizatorului.

In figura 4 se observa un apel de tip GET catre /api/cats. Putem vedea in partea de jos ca serverul a returnat statusul 200 OK, iar in corpul raspunsului se observa obiectele JSON care contin informatiile despre pisicile disponibile.

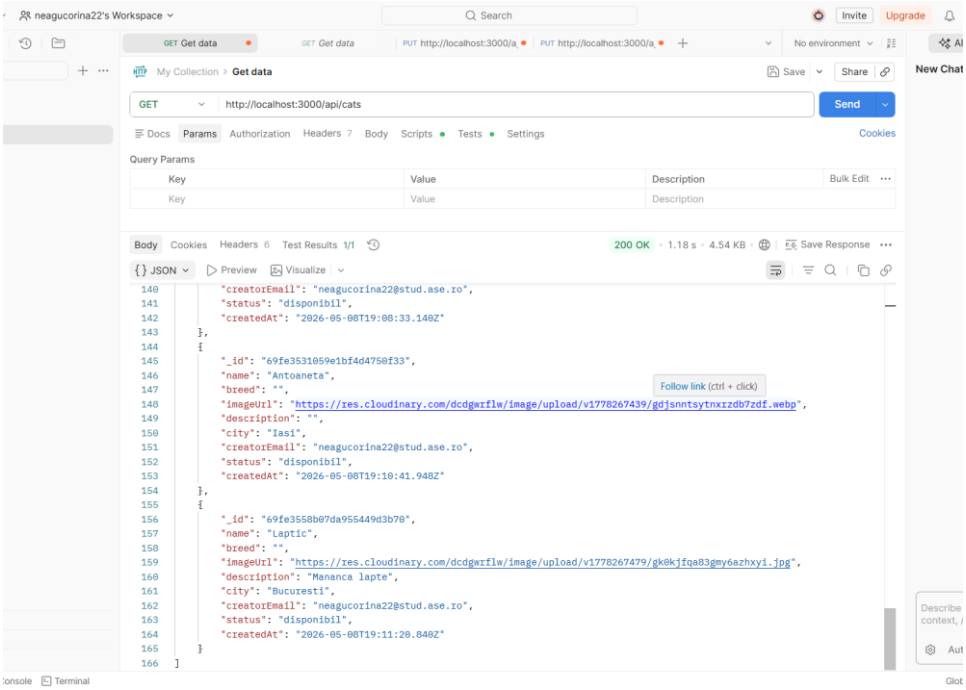

Figure 4. GET request

De asemenea, se poate observa campul imageUrl care contine link-ul catre sursa externa de stocare Cloudinary, demonstrand delegarea resurselor media catre un serviciu SaaS.

In figura 5 este testat mecanismul de Autorizare. Intrucat adaugarea unui anunt de adoptie trebuie permisa doar utilizatorilor autentificati, am incercat efectuarea unei cereri de tip POST catre endpoint-ul /api/cats fara a furniza un token de sesiune valid. Statusul HTTP 401 Unauthorized arata ca serverul a interceptat cererea prin middleware-ul de securitatea si a refuzat procesarea acesteia, deoarece necesita autentificare. Serverul returneaza un obiect de tip JSON care arata ca backend-ul functioneaza corect si identifica lipsa sesiunii, oferind un feedback clar utilizatorului.

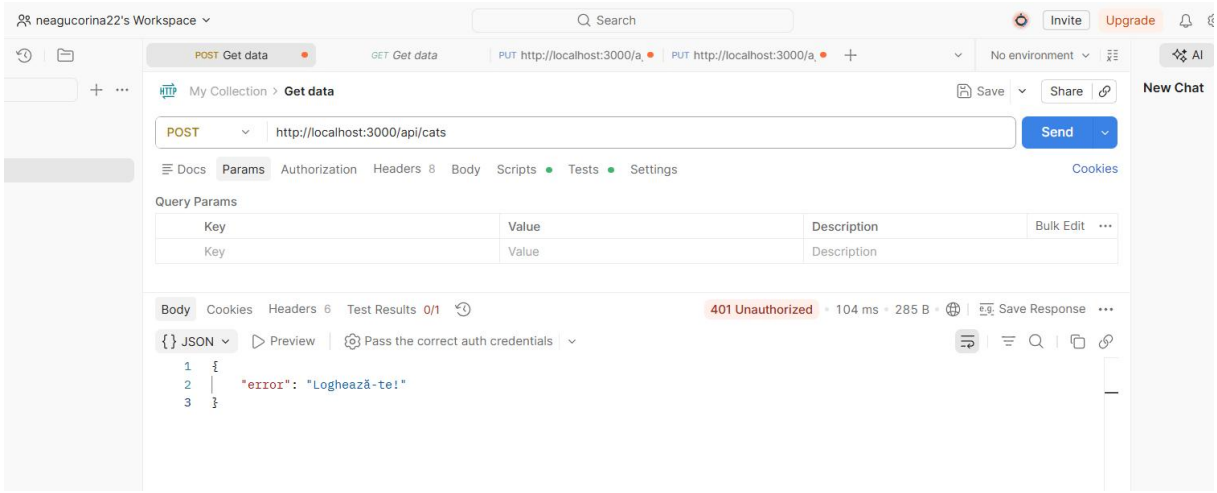

Figure 5. POST request

In figura 6 am testat functionalitatea de actualizare a informatiilor pentru un animal deja existent in baza de date. Se observa utilizarea metodei PUT catre un endpoint specific ce contine ID-ul unic al obiectului. In corpul cererii, am trimis noile date. Serverul a returnat statusul 200 OK confirmand succesul operatiunii. Raspunsul JSON primit arata ca backend-ul nu doar modifica documentul in MongoDB dar executa si logica adiacenta. Campul modified este de asemenea o dovada a interactiunii cu baza de date, confirmand ca exact un document a fost gasit si actualizat in Cloud.

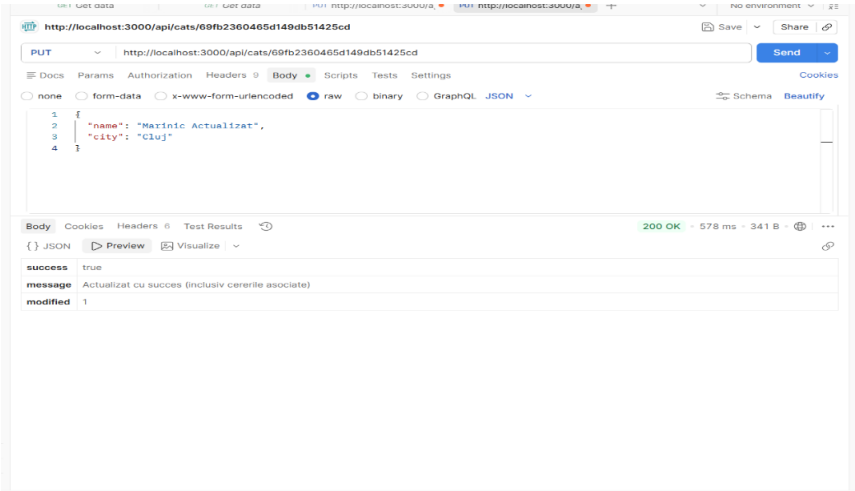

Figure 6. PUT request

In figura 7 , se testat stergerea unei inregistrari specifice folosind ID-ul sau uni generat de MongoDB utilizand ruta /api/cats/[id]. Serverul returneaza statusul 200 OK si un mesaj personalizat. Acest test confirma ca aplicatia are drepturi de scriere/stergere in clusrerul de baza de date si ca poate gestiona curatarea datelor.

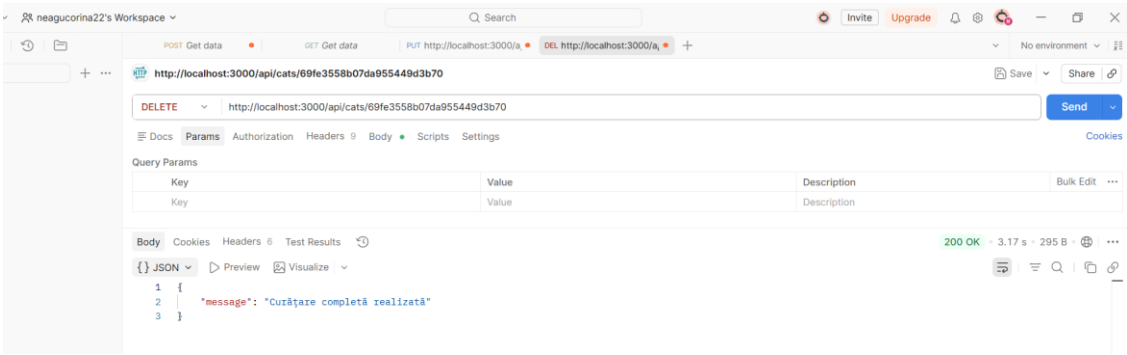

Figure 7. DELETE request

4.3. Autentificare si Autorizare

Autentificarea reprezinta procesul prin care sistemul confirma identitatea unui utilizator. Aplicatia foloseste OAuth 2.0 si externalizeaza securitatea conturilor utilizatorului prin faptul ca nu stocheaza parole in baza de date din cloud-ul MongoDB Atlas, eliminand riscurile legate de scurgerile de date. La prima logare, sistemul preia din profilul Google numele, adresa de e-mail si imaginea de profil, creand automat un document in colectia users. In figura 8 se poate observa interfata oficiala Google pentru autentificare. Odata autentificat, utilizatorul are un token de securitate criptat care este verificat la fiecare interactiune cu serverul.

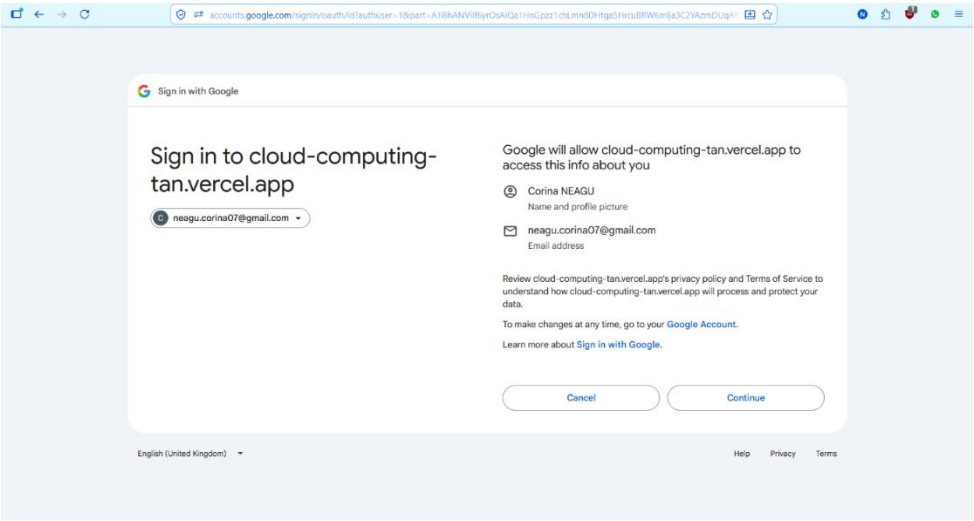

Figure 8. Interfata Google pentru autentificare

In ceea ce priveste, autorizarea si controlul accesului a fost implementata o politica bazata pe roluri. Astfel, exista doua tipuri de utilizatori: cei autentificati si cei neautentificati. Cel neautentificat are acces numai la operatii de tip Read iar orice incercare de accesare a rutelor protejate este interceptata de middleware-ul aplicatiei, NextAuth.js. Acest lucru se poate observa in figura 5. Utilizatorul autentificat primeste drepturi de scriere, stergere si modificare. Protectia tehnica este activa la nivel de Middleware pentru ca acest strat ruleaza inainte ca orice cerere sa ajunga la API, iar sistemul returneaza statusul 401 Unauthorized, protejand astfel integritatea bazei de date, asa cum este vizibil in figura 5.

5. Capturi de ecran din aplicatie

In figura 9 se poate observa homepage-ul unui utilizator care nu este autentificat in aplicatie. Pe ecran va aparea un carusel in care sunt prezentate pisicile care sunt stocate in baza de date si prezente in sistem, indiferent daca acesteau au fost sau nu adoptate. Utilizatorul are doua optiuni, de a isi crea un cont nou sau de a se autentifica in sistem.

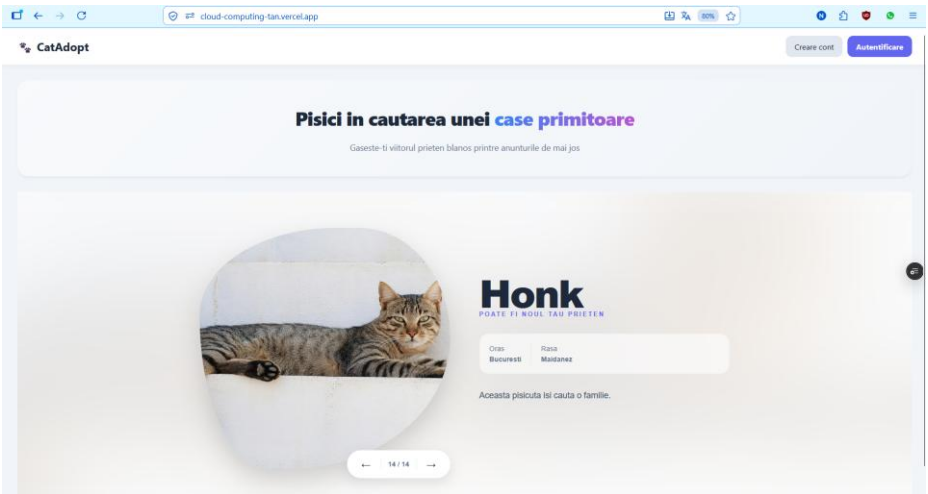

Figure 9. Homepage pentru un utilizator nelogat

Odata autentificat, utilizatorul va putea accesa lista detaliata a pisicilor din figura 10 si le va putea filtra in modul dorit. Va putea vedea cate adoptii au fost facute pana acum, pisicile care si-au gasit deja un stapan sau pisicile disponibile pentru adoptie. Mai mult decat atat, in functie de cazul fiecarei pisici utilizatorul va vedea cate un buton activ sau inactiv pentru fiecare. Butonul verde este activ si da permisiunea utilizatorului de a trimite o cerere de adoptie. Butonul gri este inactiv si anunta utilizatorul ca acea pisica a fost creata chiar de el insusi. Butonul rosu este inactiv si semnifica faptul ca pisica nu mai este disponibila pentru adoptie, desi inca este vizibila in sistem. Un buton albastru nereprezentat in figura 9 indica faptul ca o cerere a fost deja trimisa pentru pisica respectiva, blocand utilizatorul din a trimite mai multe cereri pentru aceeasi pisica.

In coltul din dreapta sus exista butonul pentru Logout, profilul utilizatorului si cele doua butoane care permit gestiunea pisicilor proprii.

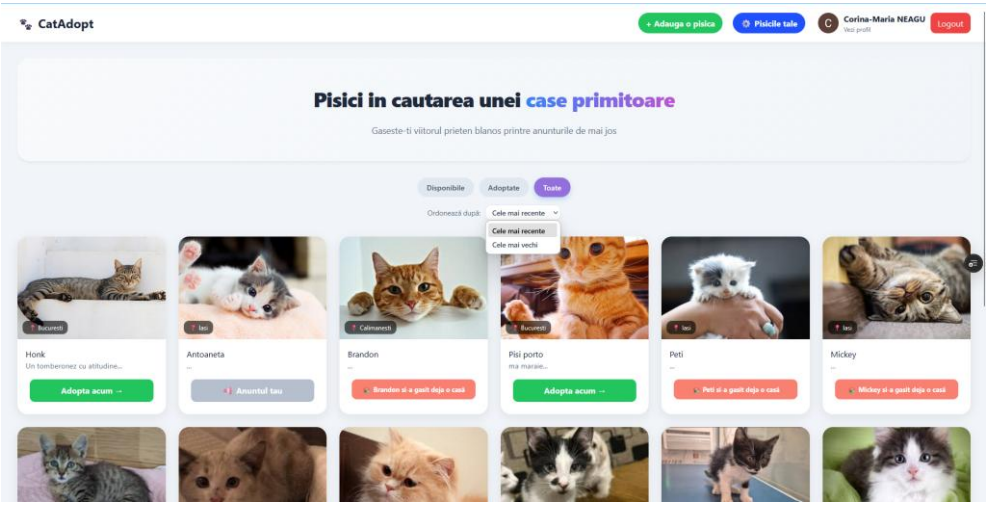

Figure 10. Homepage pentru un utilizator logat

In figura 11 se poate observa profilul utilizatorului, unde acesta are posibilitatea sa isi vada istoricul cererilor.

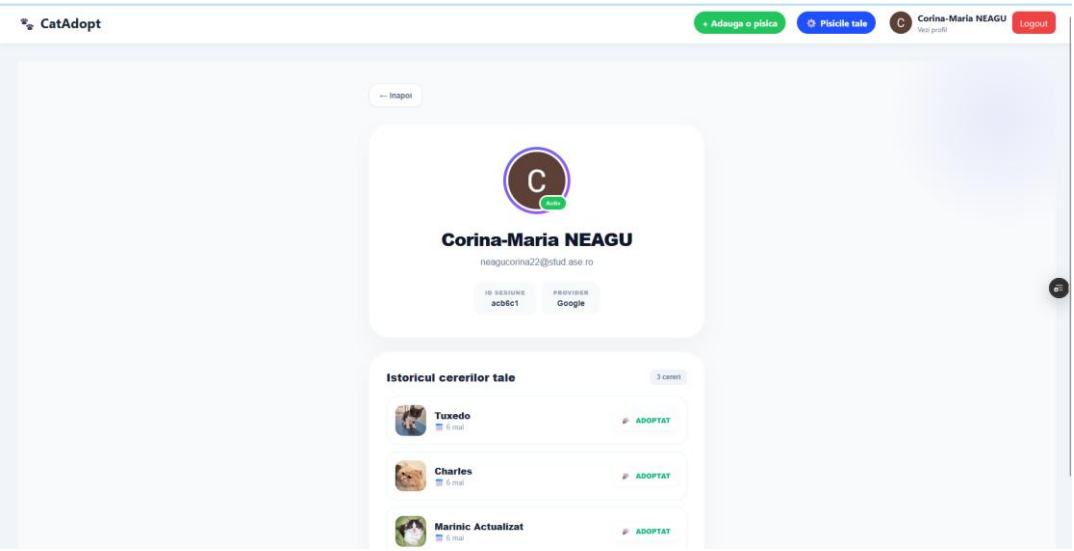

Figure 11. Profilul unui utilizator

In cazul in care utilizatorul nu a adaugat pisici in sistem si acceseaza pentru prima data butonul „Pisicile tale” va vedea ecranul din figura 12. Astfel, are posibilitatea sa fie redirectionat catre ecranul ce contine formularul de adaugare pisici.

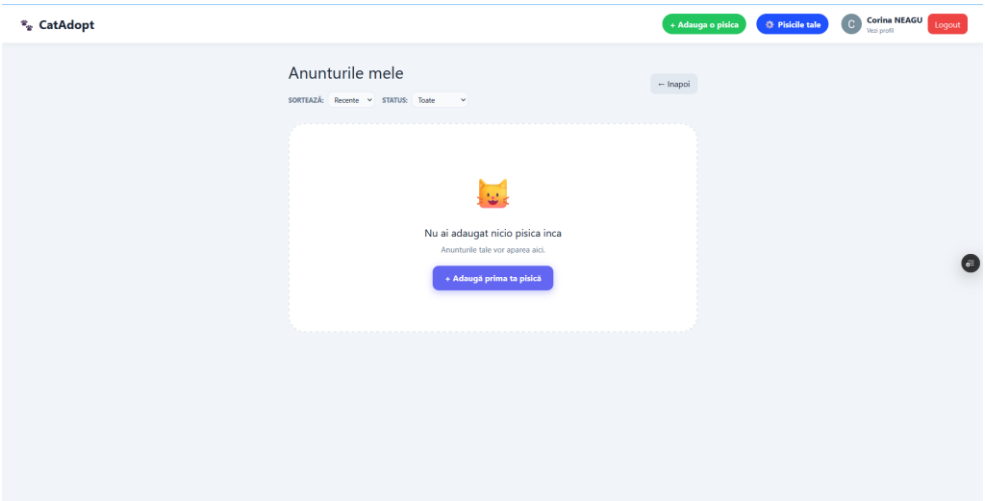

Figure 12. Pagina intermediara pentru adaugarea unei pisici

Odata ajung in formularul din figura 13, utilizatorul este obligat sa introduca date minimale despre pisica respectiva, adica numele, orasul si poza. In caz contrar acesta nu va fi lasat sa duca la bun sfarsit procesul de adaugare a pisicii in platforma intrucat vor aparea validari pentru aceste trei campuri. Daca un utilizator renunta la adaugarea unei pisici si apasa pe butonul Inapoi, progresul sau nu va fi salvat si va trebui sa adauge de la inceput detaliile pentru pisica respectiva.

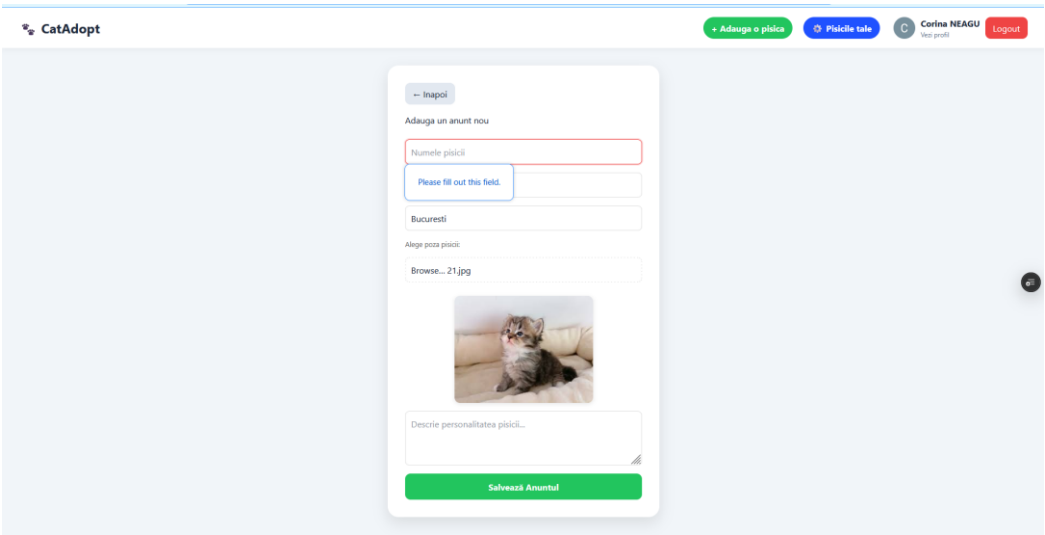

Figure 13. Formular de adaugare pisici

Cand un utilizator isi doreste sa adopte o pisica, acesta va fi nevoit sa confirme intr-un modal ca doreste sa inceapa demersurile iar un mail cu intentiile acestuia va fi trimis catre proprietarul pisicii.

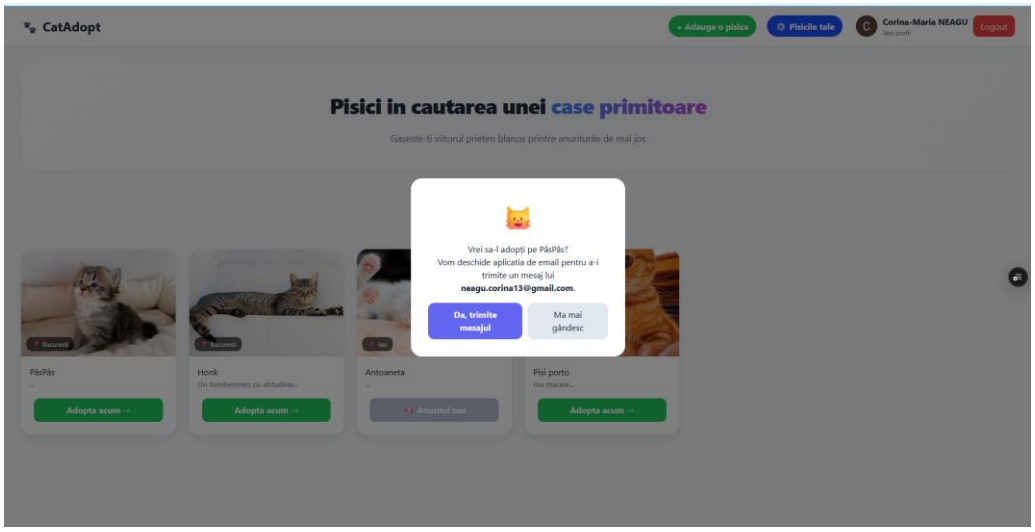

Figure 14. Modal pentru confirmarea cererii de adoptie

In figura 15 se observa mailul trimis automat de catre sistem in care proprietarul este instiintat ca pisica este dorita pentru adoptie.

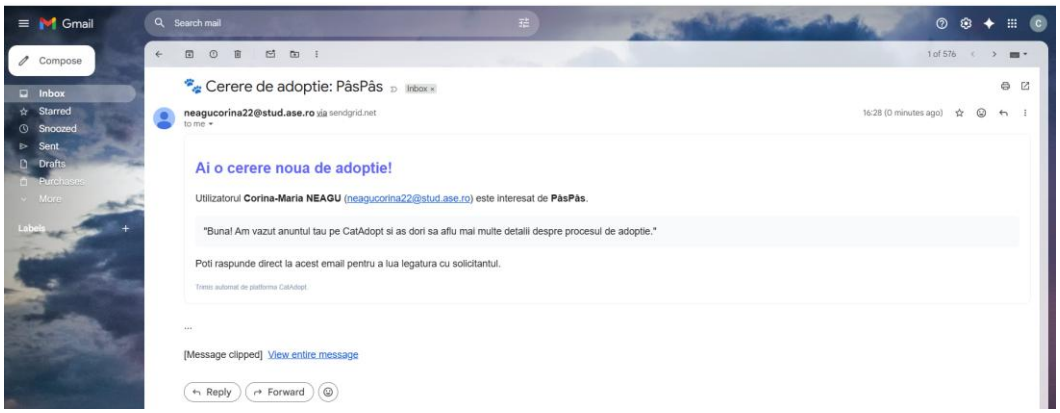

Figure 15. Cerere data pe mail prin SendGrid

In figura 16, proprietarul pisicii va intra in pagina deschisa de butonul „Pisicile tale” pentru a accepta sau a refuza o cerere de adoptie, dar si pentru a edita sau a sterge un anunt cu o pisica. De asemenea, acesta va putea sa isi filtreze anunturile in functie de vechime si de statusul acestora. Proprietarul pisicii va fi nevoit sa confirme adoptie. In cazul in care exista mai multe cereri pentru aceeasi pisica, un accept al unei cereri va duce la respingerea automata a celorlalte cereri. De asemenea, utilizatorul care a dat spre adoptie pisica nu va mai putea modifica sau sterge anuntul odata ce adoptie a fost inscrisa in baza de date. Acest lucru este posibil prin blocarea butoanelor odata ce s-a luat o decizie in privinta pisicii. In figura 17 se poate observa o adoptie incheiata cu succes si statusul unui anunt, blocand definitiv actiunile asupra acelui anunt.

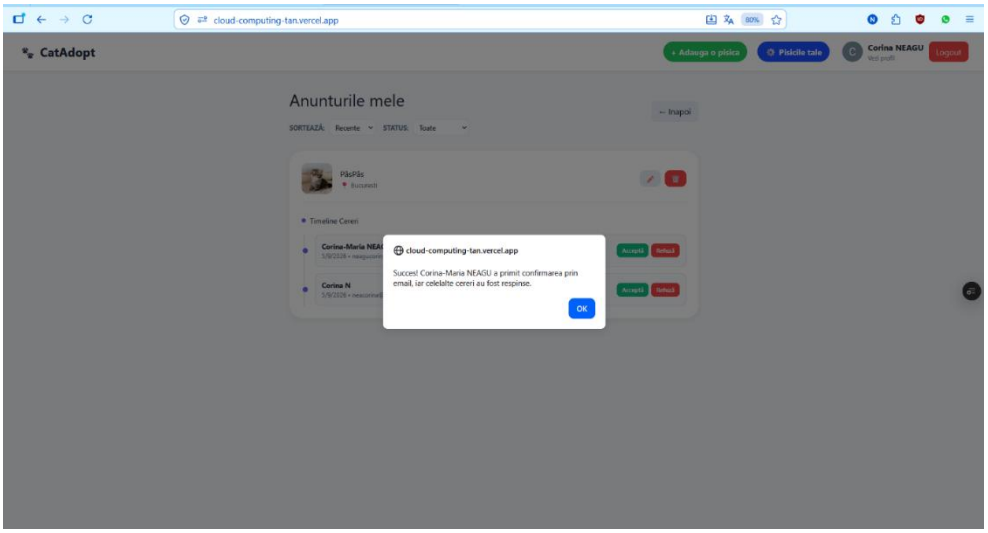

Figure 16. Ecranul de adoptie pisici

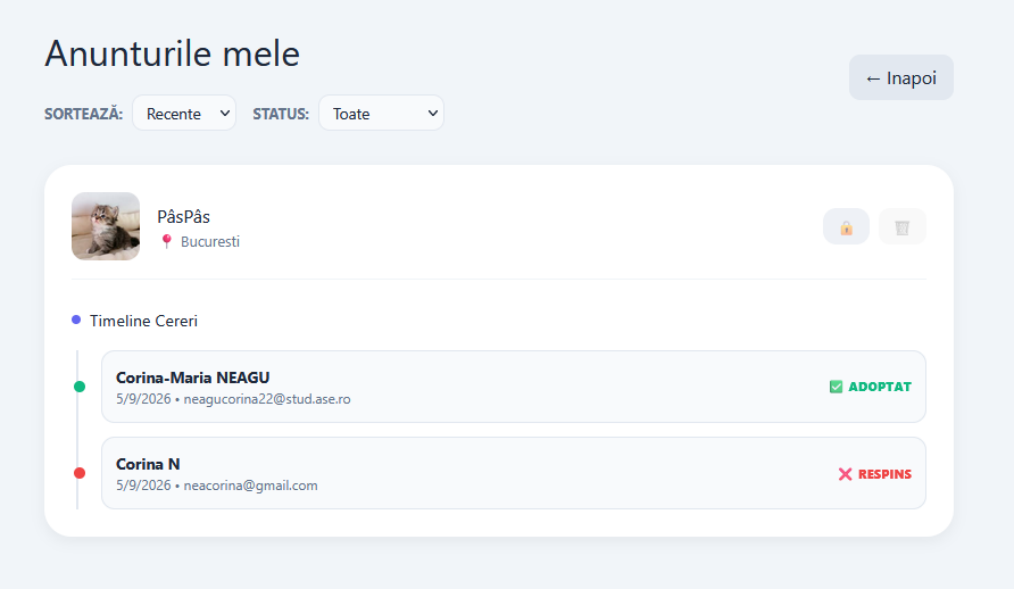

Figure 17. Adoptie confirmata

Figura 18 reprezinta confirmarea adoptiei care consta intr-un mail care va fi trimis catre utilizatorul al carei cereri a fost acceptate cu un mesaj automat si standard trimis tot prin platforma SendGrid. In cazul in care cei doi utilizatori vor sa ia legatura unul cu altul, acestia pot da pur si simplu reply la mail-urile primite deoarece platforma SendGrid permite configurarea parametrului replyTo redirectionand raspunsurile directe catre adresa de e-mail a proprietarului anuntului. Astfel, aplicatia faciliteaza comunicarea securizata intre cele doua parti, transformand e-mail-ul automat intr-un canal de dialog privat, fara a fi necesara o alta infrastructura complexa de chat in cadrul platformei.

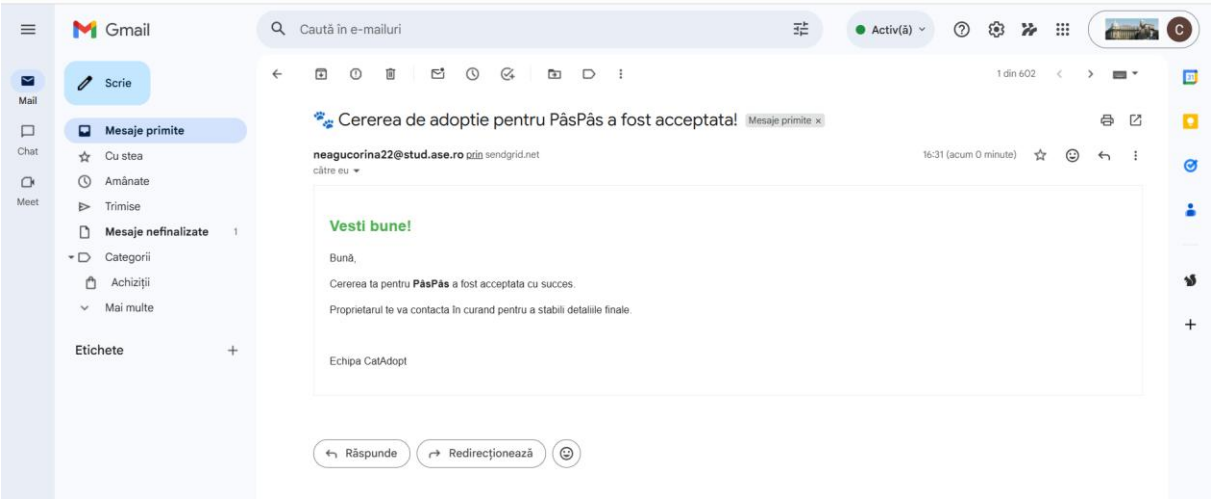

Figure 18. Confirmare prin email prin SendGrid

Lista de figuri

Figure 1. Google Cloud Console 4
Figure 2. Cloudinary Dashboard 5
Figure 3. SendGrid Dashboard 6
Figure 4. GET request 7
Figure 5. POST request 8
Figure 6. PUT request 9
Figure 7. DELETE request 9
Figure 8. Interfata Google pentru autentificare 10
Figure 9. Homepage pentru un utilizator nelogat 10
Figure 10. Homepage pentru un utilizator logat 11
Figure 11. Profilul unui utilizator 11
Figure 12. Pagina intermediara pentru adaugarea unei pisici 12
Figure 13. Formular de adaugare pisici 12
Figure 14. Modal pentru confirmarea cererii de adoptie 13
Figure 15. Cerere data pe mail prin SendGrid 13
Figure 16. Ecranul de adoptie pisici 14
Figure 17. Adoptie confirmata 14
Figure 18. Confirmare prin email prin SendGrid 15